

Article

Privacy-Preserving Byzantine-Resilient Swarm Learning for E-Healthcare

Xudong Zhu ^{1,*} , Teng Lai ¹ and Hui Li ^{2,*} 
¹ School of Information and Control Engineering, Xi'an University of Architecture and Technology, Xi'an 710055, China; 720@xauat.edu.cn

² School of Cyber Engineering, Xidian University, Xi'an 710126, China

* Correspondence: zhuxudong@xauat.edu.cn (X.Z.); lihui@mail.xidian.edu.cn (H.L.)

Abstract: An automatic medical diagnosis service based on deep learning has been introduced in e-healthcare, bringing great convenience to human life. However, due to privacy regulations, insufficient data sharing among medical centers has led to many severe challenges for automated medical diagnostic services, including diagnostic accuracy. To solve such problems, swarm learning (SL), a blockchain-based federated learning (BCFL), has been proposed. Although SL avoids single-point-of-failure attacks and offers an incentive mechanism, it still faces privacy breaches and poisoning attacks. In this paper, we propose a new privacy-preserving Byzantine-resilient swarm learning (PBSL) that is resistant to poisoning attacks while protecting data privacy. Specifically, we adopt threshold fully homomorphic encryption (TFHE) to protect data privacy and provide secure aggregation. And the cosine similarity is used to judge the malicious gradient uploaded by malicious medical centers. Through security analysis, PBSL is able to defend against a variety of known security attacks. Finally, PBSL is implemented by uniting deep learning with blockchain-based smart contract platforms. Experiments based on different datasets show that the PBSL algorithm is practical and efficient.

Keywords: blockchain-based federated learning; privacy-preserving Byzantine-resilient training; automatic medical diagnosis; healthcare



Citation: Zhu, X.; Lai, T.; Li, H.

Privacy-Preserving Byzantine-Resilient Swarm Learning for E-Healthcare. *Appl. Sci.* **2024**, *14*, 5247. <https://doi.org/10.3390/app14125247>

Academic Editor: David Megías

Received: 1 April 2024

Revised: 8 June 2024

Accepted: 10 June 2024

Published: 17 June 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In recent years, online medical diagnosis services [1] have attracted considerable interest due to their ability to overcome geographical restrictions and reduce the waiting time of seeing doctors [2,3]. To discover hidden diseases from collected medical data, deep learning (DL) has been applied to e-healthcare systems [4–6]. DL can process huge amounts of medical data and automatically find valid patterns that are too complex for human-led analysis. DL intrinsically relies on large amounts of training data. Unfortunately, in traditional healthcare systems, medical data tend to be distributed among different hospitals, and the small amount of medical data collected by individual hospitals are often insufficient to train reliable models [7,8]. Therefore, data centralization is an effective way to address the lack of training data [9]. Although the centralized model has brought many benefits, it still faces many challenges, including surge in data traffic, disclosure of local data, confidentiality of global models, and data monopolies [10].

As shown in Figure 1a, Google's federated Learning (FL) [11,12] is implemented with the help of a centralized server that aggregates all local model gradients and takes an overall average to produce a global model update. Although the problems of local data storage and local data confidentiality have been solved, the global model parameters are still processed and stored by the centralized server. Furthermore, such centralized architectures decrease fault tolerance. To solve these problems, Stefanie et al. [13] proposed swarm learning (SL), in which multiple medical centers share their local gradients through blockchain peer-to-peer networks, and the medical center that wins the consensus mecha-

nism replaces the centralized parameter server to update the global parameters according to collected gradients.

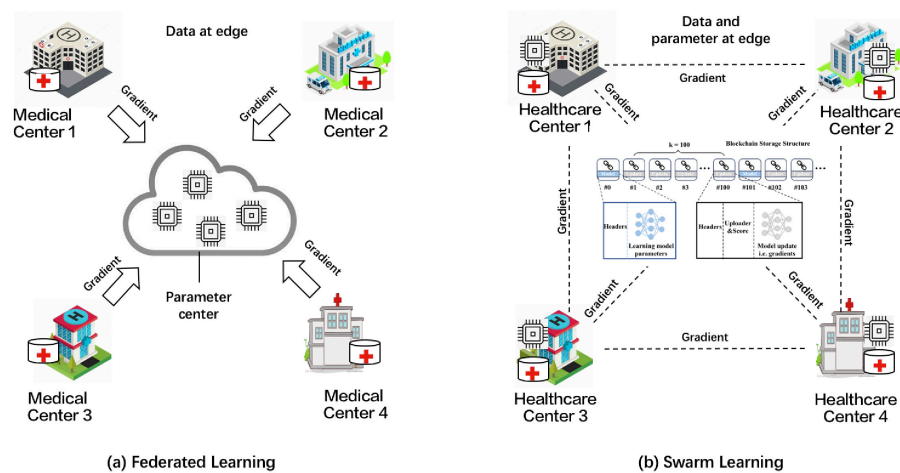


Figure 1. (a) Traditional federated learning (FL) architecture and (b) swarm learning (SL) architecture.

As shown in Figure 1b, despite the move away from centralized servers, SL still faces data security and privacy breaches. First, SL does not protect the privacy of hospital data and guarantee the confidentiality of the global model. While local model updates and global models are already recorded on blockchain, blockchain itself does not guarantee the privacy of local data and the confidentiality of the global model. For example, any curious medical center participating in swarm learning can infer the original training data of other medical centers according to the gradient and model parameters stored in the blockchain [14,15]. Secondly, in the process of gradient collection and parameter updating, the malicious behavior of dishonest medical centers can destroy the swarm learning process and poison the federated model. In particular, malicious medical centers (e.g., Byzantine attackers) can easily disrupt swarm learning systems by spreading false information over the network [16]. For example, Bagdasaryan et al. demonstrated that dishonest edge devices can poison collaborative models by replacing updated models with its exquisitely designed one [17].

Therefore, we propose a privacy-preserving Byzantine-robust swarm learning (PBSL), which ensures the confidentiality of the local gradient, the privacy of the global model, and the robustness of Byzantine attacks. Specifically, after the blockchain-based point-to-point network is established, each medical center calculates its local gradients according to the local training data. Then, the local gradients encrypted by the fully homomorphic encryption (FHE) scheme, CKKS, are broadcast to all other medical centers on the blockchain-based network. If medical centers on the blockchain reach a consensus, the winning medical center uses a proposed Byzantine attack tolerant aggregation scheme based on secure cosine similarity, which supports lossless collaborative model learning while protecting the privacy of the medical center.

In short, the main contributions of this paper include the following three aspects:

- PBSL can not only protect the privacy of local sensitive data, but also ensure the confidentiality of the global collaborative model. Using the fully homomorphic encryption (FHE) scheme CKKS, PBSL encrypts the local gradient submitted by the medical center and aggregates to generate a global collaborative model without decryption.
- PBSL can achieve swarm learning (SL) that resists Byzantine attacks. In order to resist Byzantine attacks, we propose a privacy-protecting Byzantine-robust aggregation method, which uses secure cosine similarity to punish malicious MCs.

- PBSL prototype is implemented by integrating deep learning and blockchain-based smart contract. Moreover, PBSL is tested using real medical datasets to evaluate its effectiveness. Experimental results show that PBSL is effective.

The remainder of this paper is organized as follows. We review related works in Section 2. We review some primitives in Section 3, and introduce our system model and design goals in Section 4. Then, we introduce our scheme in Section 5. Next, we present the security and performance analyses in Sections 6 and 7, respectively. Finally, we conduct a summary of our work in Section 8.

2. Related Work

In this section, we focus on related works about privacy-preserving FL, Byzantine-robust FL, blockchain-based FL, and swarm learning.

2.1. Privacy-Preserving Federated Learning

Although the training dataset is divided and stored separately, federated learning (FL) cannot protect training data privacy because the exchanged global model and local gradients still contain a large amount of sensitive information. Three popular privacy-protecting federal learning (PPFL), i.e., differential privacy (DP), homomorphic encryption (HE), and secure multi-party computation (MPC), have been proposed to address this problem.

DP-based PPFL protects the local updates against inference attacks through adding random noises [18–20]. However, it is challenging to bring a trade-off of accuracy loss. SMC-based PPFL collaboratively computes the aggregated values between multi-parties' inputs through secure multi-party computing in nature [21–26]. SMC-based PPFL has been constructed in one [27], two [28], three [29], and four [30] servers. However, although SMC, which is a cryptographic primitive, has powerful privacy, it is inefficient. HE-based PPFL uses homomorphic encryption to support aggregation operation on cipher-texts, and protects training data privacy from curious parameter servers without external learning accuracy loss [31]. EaSTFLy [32] uses (T, N) -Shamir' secret sharing and Paillier partially homomorphic encryption (PHE). BatchCrypt [33] encodes quantized gradients into signed integers and then encrypts them, achieving training speedup compared with full-precision encryption. However, most of them contain massive, complicated arithmetical operations, which brings considerable computation overhead.

In this paper, threshold fully homomorphic encryption (TFHE), which combines (T, N) -Shamir' secret sharing and CKKS fully homomorphic encryption (FHE) [34], is used as a privacy protection mechanism, which greatly reduces the computing and communication overheads.

2.2. Byzantine-Robust Federated Learning

2.2.1. Byzantine Attack

As we all know, federated learning is vulnerable to Byzantine attacks [35–37]. In order to map the specified feature to the target class, the Byzantine attacker tries to modify the classification boundary of the trained model. It has been proven that an attacker can control the entire training process and steal user privacy [16].

According to the goals of the attackers, Byzantine attacks can be divided into targeted attacks [17] and untargeted attacks [35]. Targeted attacks target only one or a few data categories in the dataset, while maintaining the accuracy of other categories. Untargeted attacks are undifferentiated attacks that aim to reduce the accuracy of all data categories. And according to the abilities of the adversaries, Byzantine attacks can also be divided into data attacks [37] and model attacks [35]. In data attacks, the attackers indirectly poison the global model by poisoning the client's local data. For example, the label-flipping attack flips the label of the normal features to the target class. And the backdoor attack is an attempt to seek a set of parameters to establish a strong link between the trigger and the target label while minimizing the impact on benign input classification. In model attacks, the attackers can directly manipulate and control the model updates of communication between the clients and the server, which directly affects the accuracy of the global model.

2.2.2. Byzantine-Robust Federated Learning

Typical FLs, such as FedSGD [38] and FedAvg, are not resistant to poisoning attacks, resulting in untrustworthy and inaccurate global models. By comparing local gradients received from different parties, Byzantine-robust federated learning (BRFL) [16,39–41] excludes abnormal gradients to protect the training model from poisoning attacks. Based on the Euclidean distance between any two gradients, Krum [16] excludes the outlier gradients. Specifically, in each training round, a candidate model would be selected. Similarly, Trim-mean [39] first sorts the received local gradients, then removes the larger and smaller gradients among them, and finally takes the mean of the remaining gradients as the global gradient. GeoMed [42] updates the FL model by selecting a gradient based on the geographic median to realize gradient aggregation. Bulyan [43] first implements the Krum-based aggregation and then updates the global model by averaging the gradient closest to the median value. Meanwhile, based on the cosine similarity between the parties' historical gradients, FoolsGold [35] sets the weights of parties to ward off Sybil attacks in FL.

However, these schemes do not consider privacy threats, which directly aggregate the global model directly in plaintext. In order to solve this problem, the FL scheme [40] using DP and SMC can not only prevent inference attack during training, but also improve model accuracy. However, because it is difficult to calculate the similarity between the encrypted gradients, the FL scheme [40] is vulnerable to malicious gradients. To this end, privacy-enhanced federated learning (PEFL), which employs HE as the underlying technology, punishes malicious parties via the logarithmic function of gradient data. However, the PEFL scheme still does not avoid malicious behaviors of the servers and inference attacks onto clients.

To develop a Byzantine-resilient, and at the same time, privacy-preserving federated learning framework, we use TFHE to protect user privacy, cosine similarity to remove malicious gradients, and blockchain to avoid malicious behavior of servers.

2.3. Blockchain-Based Federated Learning and Swarm Learning

By heavily relying on a single server, traditional FL cannot protect against a single point of failure and resist malicious attacks. As the infrastructure of Bitcoin, blockchain has attracted a lot of attention because of its decentralization and reliability. Several blockchain-based FL protocols [44–47] replace servers with blockchain, enabling privacy protection and enhancing trust among parties. BAFFLE [44] uses blockchain to avoid single points of failure and aggregates local models using smart contracts (SCs). Similarly, through blockchain smart contracts, BlockFL [46] exchanges and validates local model updates. By providing incentives associated with training samples, BlockFL can integrate more equipment and more training samples. Meanwhile, as a state-of-the-art BCFL, swarm learning (SL) [13,48] combines edge computing, blockchain networking, and federated learning while maintaining confidentiality without a central coordinator. In the e-health field, SL has been successfully applied to the diagnosis of COVID-19, tuberculosis, leukemia and lung diseases [13], skin lesion classification fairness [49], genomics data sharing [50], and risk prediction of cardiovascular events [51].

However, the above schemes impose heavy computation and communication costs on the blockchain nodes and cannot distinguish the malicious gradients. To address these issues, the blockchain-based federated learning framework with committee consensus (BFLC) [45] can effectively reduce consensus computing and prevent malicious attacks through innovative committee consensus mechanisms.

Finally, Table 1 shows a comparison of our PBSL scheme with the various FL schemes described above. The results show that our scheme can not only resist poisoning attacks and be robust to users dropping out, but also has high privacy and computational efficiency. The compared results point out that PBSL can not only resist Byzantine attacks and be robust to parties dropping out, but also protect privacy and improve computing efficiency.

Table 1. A comparative summary between our scheme and previous schemes.

Schemes	Privacy Protection	Poisoning Defense	Central Server or Blockchain	Robust to Off-Line	Lightweight Computation
SecProbe [20]	DP-based	×	Central server	✓	✓
PPDL [31]	HE-based	×	Central server	×	×
PPML [27]	MPC-based	×	Central server	✓	×
Krum [16]	×	✓	Central server	×	✓
Trim-mean [39]	×	✓	Central server	×	✓
FoolsGold [35]	×	✓	Central server	×	✓
PEFL [41]	HE-based Paillier	✓	Central server	✓	×
BAFFLE [44]	×	×	Blockchain	×	×
BlockFL [46]	×	×	Blockchain	×	×
Swarm Learning [46]	×	×	Blockchain	×	×
BFLC [45]	×	✓	Blockchain	×	✓
PBSL	HE-based CKKS	✓	Blockchain	✓	✓

3. Preliminaries

In this section, we review the basics of PBSL, namely swarm learning and full homomorphic encryption.

3.1. Swarm Learning

As the state-of-the-art BCFL, discarding the centralized server, swarm learning (SL) shares local gradients through the swarm network, and independently builds models on local data at the swarm edge nodes. SL ensures the confidentiality, privacy, and security of sensitive data with the help of blockchain. Unlike clients in traditional FL, edge nodes not only perform training tasks, but also execute transactions. The workflow of SL is as follows. A new edge node is first registered to the smart contract (SM) pre-deployed on the blockchain. Then, the swarm edge nodes perform a new training round, including downloading the global model and performing localized training. And then, all swarm edge nodes commit the local gradients to the SM through the swarm network. Finally, the SM aggregates all local gradients to create an updated model.

Suppose there are K parties $\{P_1, P_2, \dots, P_K\}$ that constitute a collaborative group, in which each party P_k holds its local dataset D_k ($k = 1, 2, 3, \dots, K$). $D = \{D_1, D_2, \dots, D_K\}$ denotes the joint dataset. Without disclosing their local sensitive data, K parties aim to cooperatively train a global general classifier $f(\mathbf{x}; \mathbf{w})$ from the complete dataset D . As illustrated in Figure 1b, in t th iteration, the party P_k downloads the global model $\mathbf{w}_{t-1}$ from the blockchain and trains the local model based on the local dataset D_k . Equation (1) describes the objective function optimized by training:

$$F(\mathbf{x}_i, \mathbf{w}, y_i) = \min_{\mathbf{w}} \mathbb{E}_{(\mathbf{x}_i, y_i) \sim \tilde{D}} L(f(\mathbf{x}_i; \mathbf{w}), y_i) \quad (1)$$

where $(\mathbf{x}_i, y_i)$ is the training sample, $L(\cdot, \cdot)$ is the empirical error function, and $\tilde{D}$ is the training sample distribution. We adopt stochastic gradient descent (SGD) to solve this minimization problem. Using $\mathbf{w}_{t-1}$ and D_k , each party P_k computes its local gradient $\nabla g_k(\mathbf{w}_t)$ via Equation (2):

$$\nabla g_k(\mathbf{w}_t) = \sum_{i=1}^{|D_k|} \frac{d}{d\mathbf{w}_t} L(f(\mathbf{x}_i^k; \mathbf{w}_{t-1}), y_i^k) \quad (2)$$

Through the consensus mechanism, the winning party performs the model update operation with Equation (3):

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \gamma \sum_{k=1}^K \zeta_i^j \nabla g_k(\mathbf{w}_t) \quad (3)$$

where $\zeta_i^j = \frac{|D_j|}{|D|}$, $\sum_{j=1}^N \zeta_i^j = 1$, and γ is the local learning rate. Once the update operation is completed, the winning party then sends the updated results through transactions to all parties.

3.2. The CKKS Scheme

As a typical fully homomorphic encryption (FHE), the Cheon–Kim–Kim–Song (CKKS) scheme [34] is a cryptographic technique in which addition and multiplication operations on plaintext are equivalent to corresponding operations on cipher-text. With the unique encoding, decoding, and rescaling mechanisms, CKKS can encrypt float numbers and vectors. CKKS consists of the following functions:

- (1) *Key Generation*: $(sk, pk, evk) \leftarrow \text{KeyGen}(1^\lambda)$. This procedure generates a public key pk for encryption, a corresponding secret key sk for decryption, and a key evk for evaluation.
- (2) *Encoding*: $m \leftarrow \text{Ecd}(\mathbf{z}, s)$. Taking an $N/2$ -dimensional vector $\mathbf{z} = (z_j)_{j \in \mathcal{T}}$ and a scaling factor s as inputs, the procedure transforms $\mathbf{z}$ to a polynomial m by encoding, i.e., $m = \sigma^{-1} \circ \pi^{-1}(\mathbf{z} \cdot s)$, where σ^{-1} and π^{-1} are the inverses of σ and π , respectively.
- (3) *Encryption*: $c \leftarrow \text{Enc}(m, pk)$. Taking the given polynomial $m \in \mathcal{R}$ and public key pk as inputs, the procedure outputs the cipher-text $c \in \mathcal{R}_{q_L}^k$.
- (4) *Addition*: $\hat{c} \leftarrow \text{Add}(c_1, c_2)$. Taking the cipher-texts c_1 and c_2 as inputs, the procedure outputs $\hat{c} = c_1 \oplus c_2$.
- (5) *Multiplication*: $\tilde{c} \leftarrow \text{Mult}(c_1, c_2, evk)$. Taking the cipher-texts c_1 and c_2 and the evaluation key evk as inputs, the procedure outputs $\tilde{c} = c_1 \otimes c_2 \in \mathcal{R}_{q_l}^k$.
- (6) *Rescaling*: $c' \leftarrow \text{RS}(c)$. Taking a cipher-text $c \in \mathcal{R}_{q_l}^k$ at level l as inputs, the procedure outputs the cipher-text $c' \leftarrow \lfloor \frac{q_{l'}}{q_l} c \rfloor$ in $\mathcal{R}_{q_{l'}}^k$, where $\lfloor x \rfloor$ represents the integer closest to the real number x .
- (7) *Decryption*: $m \leftarrow \text{Dec}(c, sk)$. Taking the cipher-text c and the secret key sk as inputs, the procedure outputs the polynomial m .
- (8) *Decoding*: $\mathbf{z} \leftarrow \text{Dcd}(m, s)$. Taking the plaintext polynomial $m \in \mathcal{R}$ as inputs, the procedure transforms $m \in \mathcal{R}$ into a vector $\mathbf{z}$ using $\mathbf{z} = \pi \circ \sigma(s^{-1} \cdot m)$.

3.3. Threshold Fully Homomorphic Encryption

To construct distributed FHE, threshold fully homomorphic encryption (TFHE) [52] has been proposed, in which each participant has a share of a secret key corresponding to an FHE public key.

Let $P = \{P_1, \dots, P_N\}$ be a set of parties. Threshold fully homomorphic encryption scheme (TFHE) is tuple of PPT algorithms $\text{TFHE} = (\text{TFHE.Setup}, \text{TFHE.Encrypt}, \text{TFHE.Eval}, \text{TFHE.PartDec}, \text{TFHE.FinDec})$ satisfying the following specifications:

- $(pk, sk_1, \dots, sk_N) \leftarrow TFHE.Setup(1^\lambda, 1^d, \mathbb{A})$: The algorithm inputs a security parameter λ , a circuit depth d , and an access structure $\mathbb{A}$, and outputs a public key pk , and secret key shares $sk_1, \dots, sk_N$.
- $ct \leftarrow TFHE.Encrypt(pk, \mu)$: The algorithm inputs a public key pk and a single-bit plaintext $\mu \in \{0, 1\}$, and outputs a cipher-text ct .
- $\hat{ct} \leftarrow TFHE.Eval(pk, C, ct_1, \dots, ct_k)$: The algorithm inputs a public key pk , a boolean circuit $C : \{0, 1\}^k \rightarrow \{0, 1\} \in C_\lambda$ of $depth \leq d$ and cipher-texts $ct_1, \dots, ct_k$ encrypted under the same public key, and outputs an evaluation cipher-text $\hat{ct}$.
- $p_i \leftarrow TFHE.PartDec(pk, ct, sk_i)$: The algorithm inputs a public key pk , a cipher-text ct and a secret key share sk_i , and outputs a partial decryption p_i related to the party P_i .
- $\hat{\mu} \leftarrow TFHE.FinDec(pk, B)$: The algorithm inputs a public key pk , a set $B = \{p_i\}_{i \in S}$ for some $S \subseteq \{P_1, \dots, P_N\}$ where we recall that we identify a party P_i with its index i , and deterministically outputs a plaintext $\hat{\mu} \in \{0, 1, \perp\}$.

And we construct $TFHE$ using the schemes, $FHE = (FHE.Setup, FHE.Encrypt, FHE.Eval, FHE.Decrypt)$ and $SS = (SS.Share, SS.Combine)$ as follows:

- $TFHE.Setup(1^\lambda, 1^d, \mathbb{A}_t)$: The algorithm inputs λ , d , and $\mathbb{A}_t \in TAS$, and generates $(fhepk, fheshk) \leftarrow FHE.Setup(1^\lambda, 1^d)$. Then, $fheshk$ is divided into shares using $(fheshk_1, \dots, fheshk_N) \leftarrow SS.Share(fheshk, \mathbb{A}_t)$.
- $TFHE.Encrypt(pk, \mu)$: The algorithm inputs pk and $\mu \in \{0, 1\}$, computes $ct \leftarrow FHE.Encrypt(pk, \mu)$, and output ct .
- $TFHE.Eval(pk, C, ct_1, \dots, ct_k)$: The algorithm inputs pk , C and $ct_1, \dots, ct_k$, computes $\hat{ct} \leftarrow FHE.Eval(C, ct_1, \dots, ct_k)$, and outputs $\hat{ct}$.
- $TFHE.PartDec(pk, ct, sk_i)$: The algorithm inputs pk , ct , and $sk_i \in \mathbb{Z}_q^n$, computes $p_i = FHE.Decode_0(sk_i, ct) + (N!)^2 \cdot e \in \mathbb{Z}_p$, and outputs p_i .
- $TFHE.FinDec(pk, B)$: The algorithm inputs pk and $\{p_i\}_{i \in S}$, and checks if $S \in \mathbb{A}$. If this is not the case, then it outputs $\perp$. Otherwise, it arbitrarily chooses a satisfying set $S' \subseteq S$ of size t and computes the Lagrange coefficients $\lambda_{i,0}^{S'}$ for all $i \in S'$. Then, it computes $\mu \leftarrow FHE.Decode_1(\sum_{i \in S'} \lambda_{i,0}^{S'})$, and outputs μ .

4. Problem Statement

In this section, we formulate our system model, threat model, and design goals.

4.1. System Model

As shown in Figure 2, our PBSL includes three entities: trusted authorities (TAs), medical centers (MCs), and blockchain (BC)-based peer-to-peer network. Each MC is equipped with a PC server, and MCs are interconnected through blockchain (BC)-based peer-to-peer networks. The three entities involved in the system are shown below.

- **TA** stands for Trusted Authority and is responsible for generating initial parameters and public/secret key pairs and distributing them to MCs.
- **MCs** = $\{MC_1, MC_2, \dots, MC_K\}$ represents K medical centers participating in swarm learning. In our system, each $MC_k \in MCs$ trains the model using its local medical dataset, encrypts the gradient, shares the encrypted gradient to the blockchain, and cooperatively trains with other MCs to achieve a more accurate global model.
- **BC** represents a blockchain platform made up of self-organizing networks of interconnected medical centers (MCs). Every node (i.e., MCs) on blockchain has the opportunity to aggregate the global model, so BC can replace a central aggregator. Smart contract (SC) deployed on the blockchain homomorphically aggregates the encrypted gradients to obtain the global collaboration model.

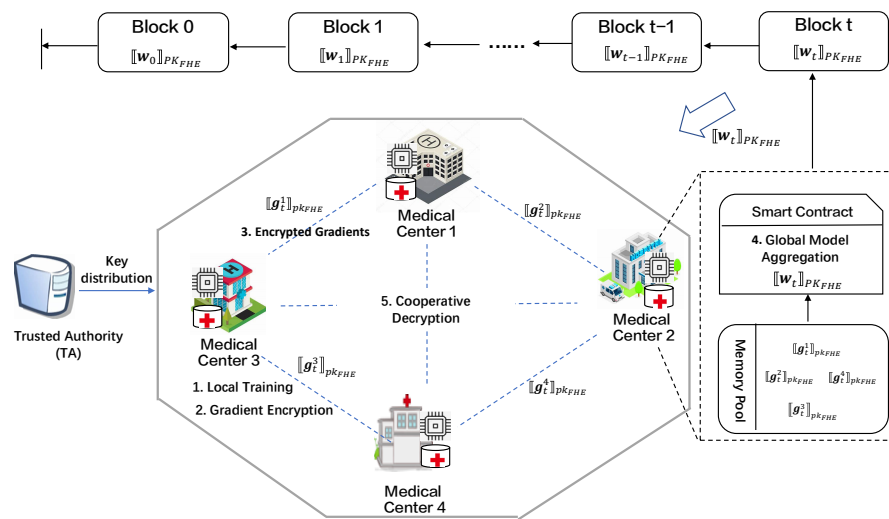


Figure 2. System model of PBSL.

4.2. Threat Model

TA is a trusted third party. Considering the autonomy of MCs in swarm learning, we select the “curious-and-malicious” threat model, which is weaker than the “honest-and-curious” threat model. Specifically, curious MCs are honest in uploading true gradients trained on their local datasets, while attempting to obtain more information from gradients shared by other MCs stored on the blockchain, thus compromising users’ data privacy. In order to reduce the global model accuracy, malicious MCs can perform malicious operations or deliberately abandon local training. The potential threats to our PBSL are as follows:

- **Privacy:** Since local gradients contain MCs’ private information, if the MCs directly upload the local gradients in plaintext, the adversaries can infer the private information of honest MCs, which leads to data leakage of MCs.
- **Byzantine Attacks:** Byzantine attack refers to the arbitrary behavior of the system participant who does not follow the designed protocol. For example, a Byzantine MC randomly sends a vector in place of the local gradient, or mistakenly aggregates the local gradient of all MCs.
- **Poisoning Attacks:** Poison attack refers to various malicious behaviors launched by the MCs. For example, an MC flips the label of the sample and uploads the toxic gradient.

4.3. Design Goals

Under the system and threat models mentioned above, aiming to design a privacy-preserving swarm learning (PBSL) scheme, the proposed scheme should provide confidentiality and privacy guarantees, resist Byzantine attacks, and reduce computational overheads. And our scheme should achieve the same precision as classical swarm learning (SL). Specifically, the following goals are achieved:

- **Confidentiality:** Guaranteeing the confidentiality of the global model stored in the blockchain. Specifically, after swarm learning, only MCs can access the global model stored in the blockchain.
- **Privacy:** Guaranteeing the privacy of each MCs’ local sensitive information. Specifically, during swarm learning, every MCs’ local training data cannot be leaked to other MCs.
- **Robustness:** Detecting malicious MCs and discarding their local gradients during swarm learning. Specifically, a mechanism is constructed to identify malicious medical centers and discard their submitted local model gradients.
- **Accuracy:** While protecting data privacy and resisting malicious attacks, the model accuracy of PBSL training should be the same as that of the original SL training.

- **Efficiency:** Reducing the high computational and communication cost of encryption operations in a privacy-preserving SL scheme.
- **Tolerance to MC drop-out:** Guaranteeing the privacy and convergence even if up to D MCs are delayed or dropped.

5. Our Privacy-Preserving Swarm Learning

As shown in Figure 3, our PBSL described in this section mainly includes four stages: (I) system initialization, (II) local computation, (III) model aggregation, and (IV) global model decryption and updating.

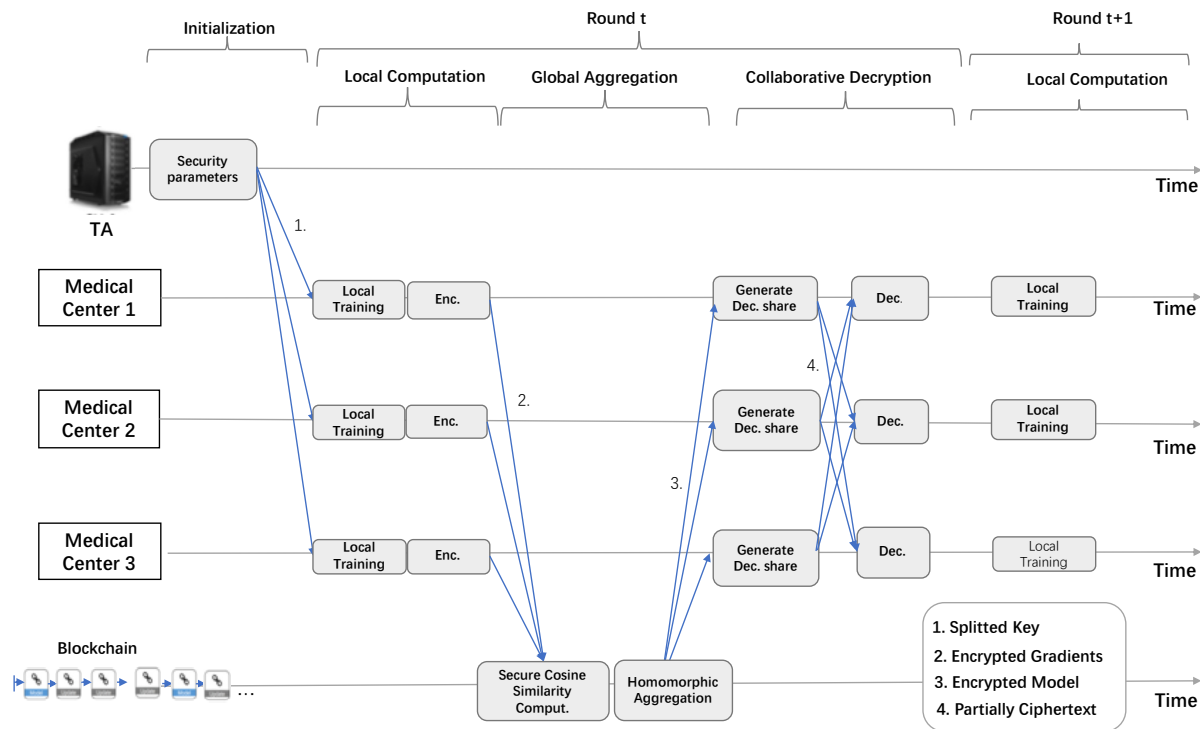


Figure 3. An overview of proposed PBSL.

Firstly, TA sets initialization parameters, generates key pairs $\langle sk_{FHE}, pk_{FHE} \rangle$, divides sk_{FHE} into K shares through Shamir's secret sharing, and distributes K shares to K MCs, respectively.

Then, MCs train their model on its local dataset and encrypt their gradients by fully homomorphic encryption technology CKKS [34], which are broadcasted to the blockchain-based network.

Furthermore, the MC that wins the consensus mechanism uses a cosine similarity-based strategy to punish malicious gradient vectors, obtain the encrypted global model through homomorphically aggregating the encrypted local gradients, and stores the encrypted collaborative learning results on the blockchain.

Finally, MCs obtain the encrypted global models from the blockchain and cooperatively decrypt the encrypted global models to obtain the global models. We summarize PBSL in Algorithm 1, and illustrate the details of each process in the following. Table 2 lists the description of notations in our PBSL.

Table 2. Summary of notations in PBSL.

Notation	Implications
K	the number of medical centers
m	the threshold of Sharmir secret sharing
$\mathbf{MC}_1, \dots, \mathbf{MC}_K$	a set of K medical centers
pk_k^{psu}, sk_k	pseudo public key and secret key of medical center $\mathbf{MC}_k$
α	the security parameter
pk_{FHE}, sk_{FHE}	the keys of CKKS cryptosystem
sk_{FHE}^k	the distributed secret keys splitted from sk_{FHE}
key_{kl}	the symmetric encryption key between medical center $\mathbf{MC}_k$ and $\mathbf{MC}_l$
$Enc_{key_{kl}}(\cdot), Dec_{key_{kl}}(\cdot)$	the symmetric key encryption or decryption algorithm using key_{kl}
$\mathbf{w}_t$	the weight of the global model after the $t - 1$ -th iteration of training
$\llbracket \cdot \rrbracket_{pk_{FHE}}$	the cipher-text generated by CKKS algorithm
$\mathcal{V}$	the size of decryption set DS

Algorithm 1: PBSL

Input: K medical centers $\{\mathbf{MC}_1, \dots, \mathbf{MC}_K\}$ with local training datasets

$D = \{D_1, \dots, D_K\}$, learning rate γ , and iterations e .

Output: Global model $\mathbf{w}$.

- 1: $\mathbf{MC}_1, \dots, \mathbf{MC}_K$ form a blockchain-based P2P network, where $\mathbf{MC}_k$ has pseudo public key pk_k^{psu} and secret key sk_k
- 2: TA generates $\langle sk_{FHE}, pk_{FHE} \rangle$, splits sk_{FHE} into K parts, and distributes sk_{FHE}^k to $\mathbf{MC}_k$, where $k \in \{1, 2, \dots, K\}$.
- 3: $\mathbf{MC}_T$ as task publisher broadcasts the SL task by deploying an SC over the network, where SC specifies hyper-parameters: local learning rate α , local batch size b , and initial global model $\mathbf{w}_0$.
- 4: Each $\mathbf{MC}_k$ publishes $\langle pk_k^{psu}, proof_{sk_k} \rangle$ to SC, and SC verifies $proof_{sk_k}$ and returns the hyper-parameters to $\mathbf{MC}_k$ if the verification passes.
- 5: **for** $t \in [1, e]$ *and accuracy of the learned model is not up to standard* **do**
 - for each** $k \in \{\mathbf{MC}_1, \dots, \mathbf{MC}_K\}$ **do**
 - Get global model $\llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}}$ from blockchain ;
 - $\llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}} \leftarrow \text{Local_Computation}(D_k, b, \alpha, r)$;
 - Broadcast the message $msg_k: M_t^k \leftarrow (pk_k^{psu}, \llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}}, t)$;
 - end**
 - Each $\mathbf{MC}_k$ competes to solve PoW problem ;
 - if** $\mathbf{MC}_k$ *wins* **then**
 - Retrieve $\{\llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}}\}_{k=1}^K$ from its memory pool ;
 - $\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} \leftarrow \text{Global_Aggregation}(\{\llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}}\}_{k=1}^K)$;
 - $\llbracket \mathbf{w}_t \rrbracket_{pk_{FHE}} \leftarrow \llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}} - \gamma \llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}}$;
 - Create $B_t = (\llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}}, [M_t^1, M_t^2, \dots, M_t^K])$, and appends B_t to the blockchain;
 - end**
- end**

5.1. System Initialization**5.1.1. Blockchain P2P Network Construction**

$\mathbf{MCs} = \{\mathbf{MC}_1, \mathbf{MC}_2, \dots, \mathbf{MC}_K\}$ form blockchain-based peer-to-peer network. $\mathbf{MCs}$ have pseudonymity, i.e., pseudo public keys $pk_1^{psu}, pk_2^{psu}, \dots, pk_K^{psu}$, and their corresponding secret keys $sk_1, sk_2, \dots, sk_K$ are privately kept, respectively.

5.1.2. Cryptosystem Initialization

Firstly, **TA** generates a key pair $\langle sk_{FHE}, pk_{FHE} \rangle \leftarrow KeyGen(1^\lambda)$. **TA** sets a security parameter λ , and chooses a power-of-two $M = m(\lambda, q_L)$, an integer $h = h(\lambda, q_L)$, an integer $P = P(\lambda, q_L)$, and a real value $\sigma = \sigma(\lambda, q_L)$. Then, **TA** samples $s \leftarrow \mathcal{HWT}(h)$, $a \leftarrow \mathcal{R}_{q_L}$, and $e \leftarrow \mathcal{DG}(\sigma^2)$, where $\mathcal{HWT}(h)$ is the set of signed binary vectors whose Hamming weight is exactly h and $\mathcal{DG}(\sigma^2)$ samples a vector in $\mathbb{Z}^N$ by drawing its coefficient independently from the discrete Gaussian distribution of variance σ^2 . The private key is set as $sk_{FHE} \leftarrow (1, s)$ and the public key as $pk_{FHE} \leftarrow (b, a) \in \mathcal{R}_{q_L}^2$ where $b \leftarrow -as + e \pmod{q_L}$.

Secondly, **TA** randomly divides sk_{FHE} into K parts. **TA** computes $\delta = \eta \cdot \lambda$, s.t., $\eta \cdot \lambda \equiv 1 \pmod{N^2}$, sets a threshold m , s.t., $(m - 1) \leq N$, and defines a polynomial function of secret sharing protocol $f(x) = \lambda + \sum_{i=1}^m a_i x^i$, where $a_1, a_2, \dots, a_{m-1} \in_R \mathbb{Z}_\lambda^*$. **TA** divides sk_{FHE} into K parts through calculating $f(i)$. **TA** broadcasts $\langle g, pk_{FHE}, \eta \rangle$ to all **MCs** through blockchain-based network.

Finally, each $\mathbf{MC}_k \in \mathbf{MC}$ asks **TA** for its own private key $sk_{FHE}^k = f(k)$.

5.1.3. Registration

To participate in swarm learning, $\mathbf{MC}_k \in \mathbf{MCs}$ need to register to smart contract (SC) published on the blockchain, which specifies the entire deep learning task. Each $\mathbf{MC}_k$ constructs a zero-knowledge proof $proof_{sk_k}$ of its private key sk_k . The $proof_{sk_k}$ is defined as $\langle c, r \rangle$, where $v \in_R \mathbb{Z}_q$, $t = g^v$, $c = H(g, pk_k^{psu}, t)$, and $r = (v - c \cdot sk_k) \pmod{q}$. Then, $\mathbf{MC}_k$ submits “Registration” transaction $\langle pk_k^{psu}, proof_{sk_k} \rangle$ to the blockchain. And then, each $\mathbf{MC}_k$ on the blockchain verifies $proof_{sk_k}$, i.e., $proof_{sk_k} \stackrel{?}{=} H(g, pk_k^{psu}, g^r (pk_k^{psu})^c)$ to prevent an adversary from registering itself with pk_k^{psu} copied from an honest $\mathbf{MC}_l$. If the verification is passed, $\mathbf{MC}_k$ is registered as a member of the learning group and stores its pk_k^{psu} to the blockchain.

And in order to protect public broadcast channels, we can construct the symmetric encryption key $key_{kl} = (pk_k^{psu})^{sk_l} = (pk_l^{psu})^{sk_k}$ and use the symmetric key encryption algorithm $Enc_{key_{kl}}(\cdot)$ to encrypt the traffic data between $\mathbf{MC}_k$ and $\mathbf{MC}_l$.

5.1.4. Parameter Initialization

In order to initialize the system parameters, a randomly selected $\mathbf{MC}_l$ sets the initial parameters $\mathbf{p} = (\eta, b, \mathbf{w}_0)$, where η is the learning rate, b is batch size, and $\mathbf{w}_0$ is initial model parameters. Then, $\mathbf{MC}_l$ broadcasts the encrypted parameters $\overline{\mathbf{p}}_{l \rightarrow k} = Enc_{key_{lk}}(\mathbf{p})$ to all other $\mathbf{MC}_k$ ($k \neq l$). Upon receiving $\overline{\mathbf{p}}_{l \rightarrow k}$ from $\mathbf{MC}_l$, $\mathbf{MC}_k$ decrypts $\overline{\mathbf{p}}_{l \rightarrow k}$ to obtain $\mathbf{p} = Dec_{key_{lk}}(\overline{\mathbf{p}}_{l \rightarrow k})$. Finally, all **MCs** obtained uniform initial parameters $\mathbf{p}$.

5.2. Local Computation

Algorithm 2 describes the procedure of **Local_Computation** in detail. In a local computation procedure, there are four parts: local model training, normalization, encryption, and sharing. The four parts are described in detail below.

5.2.1. Local Model Training

In the t th iteration, each $\mathbf{MC}_k$ obtains the encrypted global model $\llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}}$ from the latest block B_{t-1} on the blockchain. Then, each $\mathbf{MC}_k$ calls **Collab_Decryption**($\cdot$) to obtain the local model $\mathbf{w}_{t-1}$. Finally, using D_k and $\mathbf{w}_{t-1}^k$, each **MC** computes the local gradient $\mathbf{g}_t^k$, as shown in Equation (4):

$$\mathbf{g}_t^k = \nabla L(\mathbf{w}_{t-1}^k, D_k) \quad (4)$$

where $L(\mathbf{w}_{t-1}^k, D_k)$ represents the experical loss function and ∇ represents the derivation operation.

Algorithm 2: Local_Computation

Input: $\mathbf{MC}_k$ with local training dataset D_k , learning rate α , size of batch b , iteration number r .

Output: Encrypted local gradient $\llbracket \tilde{\mathbf{g}}^k \rrbracket_{pk_{FHE}}$.

- 1: Get the encrypted global model $\llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}}$ from blockchain ;
- 2: $\mathbf{w}_{t-1} \leftarrow \text{Collab_Decryption}(\llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}}, sk_{FHE}^k, \{key_{kj}\}_{j=1}^K)$
- 3: $\mathbf{w}_0 \leftarrow \mathbf{w}_{t-1}$;
- 4: **for** each $i \in [1, r]$ **do**
 - Randomly sample a batch D_r from D ;
 - $\mathbf{g}_i^k \leftarrow \nabla L(D_r, \mathbf{w}_{i-1})$;
 - $\mathbf{w}_i \leftarrow \mathbf{w}_{i-1} - \alpha \mathbf{g}_i^k$;
 - $\mathbf{g}^k \leftarrow \mathbf{g}_i^k$
- 5: $\tilde{\mathbf{g}}^k \leftarrow \frac{\mathbf{g}^k}{|\mathbf{g}^k|}$;
- 6: Get encrypted gradient $\llbracket \tilde{\mathbf{g}}^k \rrbracket_{pk_{FHE}}$ using pk_{FHE} ;

5.2.2. Normalization

In order to directly apply our cosine-similarity-based aggregation rule to ciphertext and mitigate the effects of malicious gradients with a larger magnitudes, these local gradients $\{\mathbf{g}_t^k\}_{k=1}^K$ need to be normalized. For each $\mathbf{MC}_k$, we use Equation (5) to normalize the local gradients $\mathbf{g}_t^k$:

$$\tilde{\mathbf{g}}_t^k = \frac{\mathbf{g}_t^k}{|\mathbf{g}_t^k|} \quad (5)$$

where $\tilde{\mathbf{g}}_t^k$ is the unit vector.

5.2.3. Encryption

To ensure the confidentiality of local model updates, as shown in Algorithm 3, each $\mathbf{MC}_k$ encrypts $\tilde{\mathbf{g}}_t^k$ using pk_{FHE} . Because gradients are signed the floating-point vector, gradients are encrypted using the CKKS scheme [34] instead of the Paillir scheme [53] with high computation overheads. $\tilde{\mathbf{g}}_t^k$ is treated as vectors and encrypted using pk_{FHE} .

Algorithm 3: Global_Aggregation

Input: Encrypted local gradients $\{\llbracket \tilde{\mathbf{g}}^k \rrbracket_{pk_{FHE}}\}_{k=1}^K$.

Output: Encrypted global model $\llbracket \tilde{\mathbf{g}} \rrbracket_{pk_{FHE}}$.

- 1: $\llbracket \tilde{\mathbf{g}} \rrbracket_{pk_{FHE}} \leftarrow \frac{1}{K} \sum_{k=1}^K \llbracket \tilde{\mathbf{g}}^k \rrbracket_{pk_{FHE}}$;
- 2: **for** each $k \in \{\mathbf{MC}_1, \dots, \mathbf{MC}_K\}$ **do**
 - $\llbracket cs^k \rrbracket_{pk_{FHE}} \leftarrow \llbracket \tilde{\mathbf{g}} \rrbracket_{pk_{FHE}} \odot \llbracket \tilde{\mathbf{g}}^k \rrbracket_{pk_{FHE}}$;
 - $\llbracket S^k \rrbracket'_{pk_{FHE}} \leftarrow \text{Max}(\llbracket cs^k \rrbracket_{pk_{FHE}} + \llbracket 1 \rrbracket_{pk_{FHE}}) \cdot \frac{1}{2}, \llbracket \frac{1}{2} \rrbracket_{pk_{FHE}}, d)$;
 - $\llbracket S^k \rrbracket_{pk_{FHE}} \leftarrow 2 \cdot \llbracket S^k \rrbracket' - \llbracket 1 \rrbracket_{pk_{FHE}}$;
- 3: $\llbracket \tilde{\mathbf{g}} \rrbracket_{pk_{FHE}} \leftarrow \frac{\sum_{k=1}^K \llbracket S^k \rrbracket_{pk_{FHE}} \cdot \llbracket \tilde{\mathbf{g}}^k \rrbracket_{pk_{FHE}}}{\sum_{k=1}^K \llbracket S^k \rrbracket_{pk_{FHE}}}$;

5.2.4. Message Encapsulation and Sharing

Each $\mathbf{MC}_k$ encapsulates pk_k^{psu} , $\llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk}$, and timestamp t into the following message:

$$\mathbb{M}_t^k = (pk_k^{psu}, \llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk}, t) \quad (6)$$

and broadcasts $\mathbb{M}_t^k$ to other $\mathbf{MC}$ s.

5.3. Global Aggregation

5.3.1. PoW Competition

In order to compete for the right to append blocks to the blockchain, all **MC**s that make up the blockchain execute any consensus algorithm (e.g., PoW). The only winner **MC**_{*j*} has the right to construct a block and add it to the blockchain.

5.3.2. Homomorphic Aggregation

In the *t*th iteration, the winner **MC**_{*j*} is responsible for aggregating all the encrypted gradients stored in the memory pool. **MC**_{*j*} decodes $\{\mathbb{M}_t^1, \mathbb{M}_t^2, \dots, \mathbb{M}_t^K\}$ and homomorphically aggregates encrypted gradients to generate the encrypted global model $\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}}$ without the need for decryption by the following equation:

$$\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} = \frac{1}{K} \sum_{k=1}^K \llbracket \mathbf{g}_t^k \rrbracket_{pk_{FHE}} \quad (7)$$

5.3.3. Secure Cosine Similarity Computation

Then, we compute the cosine similarity cs_t^k between two gradients $\tilde{\mathbf{g}}_t = [p_1, p_2, \dots, p_n]$ and $\mathbf{g}_t^k = [q_1, q_2, \dots, q_n]$. And the encrypted gradients $\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}}$ and $\llbracket \mathbf{g}_t^k \rrbracket_{pk_{FHE}}$ are obtained as $\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} = Enc_{pk_{FHE}}([p_1, p_2, \dots, p_n])$ and $\llbracket \mathbf{g}_t^k \rrbracket_{pk_{FHE}} = Enc_{pk_{FHE}}([q_1, q_2, \dots, q_n])$. The encrypted cosine similarity $\llbracket cs_t^k \rrbracket_{pk_{FHE}}$ between $\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}}$ and $\llbracket \mathbf{g}_t^k \rrbracket_{pk_{FHE}}$ can be transformed to the inner product, which is described as follows:

$$\begin{aligned} \llbracket cs_t^k \rrbracket_{pk_{FHE}} &= \llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} \odot \llbracket \mathbf{g}_t^k \rrbracket_{pk_{FHE}} \\ &= \llbracket p_1 q_1 + p_2 q_2 + \dots + p_n q_n \rrbracket_{pk_{FHE}} \end{aligned}$$

If $cs_t^k < 0$, it means that the angle between $\tilde{\mathbf{g}}_t^k$ and $\tilde{\mathbf{g}}_t$ is greater than 90° , that is, $\tilde{\mathbf{g}}_t^k$ has a negative impact on $\tilde{\mathbf{g}}_t$. The trust score S_t^k of **MC**_{*j*} is defined by Equation (7) to punish the malicious gradients

$$S_t^k = ReLU(cs_t^k) \quad (8)$$

Equation (8) gives the definition of the *ReLU* function.

$$ReLU(a) = \begin{cases} a, & \text{if } a > 0 \\ 0, & \text{if } a \leq 0 \end{cases}$$

According to the previous work [54], the cipher-text homomorphism operation of *ReLU* function is given below:

$$\begin{aligned} \llbracket S_t^k \rrbracket'_{pk_{FHE}} &= ReLU'(\llbracket cs_t^k \rrbracket_{pk_{FHE}}) \\ &= \begin{cases} \frac{1}{2}(\llbracket cs_t^k \rrbracket_{pk_{FHE}} + \llbracket 1 \rrbracket_{pk_{FHE}}), & \text{if } \frac{1}{2}(\llbracket cs_t^k \rrbracket_{pk_{FHE}} + \llbracket 1 \rrbracket_{pk_{FHE}}) > \llbracket \frac{1}{2} \rrbracket_{pk_{FHE}} \\ \llbracket \frac{1}{2} \rrbracket_{pk_{FHE}}, & \text{if } \frac{1}{2}(\llbracket cs_t^k \rrbracket_{pk_{FHE}} + \llbracket 1 \rrbracket_{pk_{FHE}}) \leq \llbracket \frac{1}{2} \rrbracket_{pk_{FHE}} \end{cases} \end{aligned}$$

To compare the cipher-text of $\frac{1}{2}(\llbracket cs_t^k \rrbracket_{pk_{FHE}} + \llbracket 1 \rrbracket_{pk_{FHE}})$ and $\llbracket \frac{1}{2} \rrbracket_{pk_{FHE}}$, the numerical method for the comparison of homomorphic ciphers in the range of $[0, 1]$ is called [54]. The encrypted score $\llbracket S_t^k \rrbracket_{pk_{FHE}}$ is set by

$$\llbracket S_t^k \rrbracket_{pk_{FHE}} = 2 \cdot \llbracket S_t^k \rrbracket'_{pk_{FHE}} - \llbracket 1 \rrbracket_{pk_{FHE}} \quad (9)$$

5.3.4. Model Aggregation and Updating

In the *t*th epoch, the winner **MC**_{*j*} homomorphically aggregates the local gradients weighted by their trust score according to the following equation:

$$\begin{aligned} \llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} &= \frac{\sum_{k=1}^K \llbracket S_t^k \rrbracket_{pk_{FHE}} \cdot \llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}}}{\sum_{k=1}^K \llbracket S_t^k \rrbracket_{pk_{FHE}}} \\ &= \frac{\sum_{k=1}^K ReLU(\llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} \odot \llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}}) \cdot \llbracket \tilde{\mathbf{g}}_t^k \rrbracket_{pk_{FHE}}}{\sum_{k=1}^K \llbracket S_t^k \rrbracket_{pk_{FHE}}} \end{aligned}$$

and subsequently updates the global model:

$$\llbracket \mathbf{w}_t \rrbracket_{pk_{FHE}} = \llbracket \mathbf{w}_{t-1} \rrbracket_{pk_{FHE}} - \gamma \llbracket \tilde{\mathbf{g}}_t \rrbracket_{pk_{FHE}} \quad (10)$$

5.3.5. Construction of New Block

$\mathbf{MC}_k$ constructs the new block B_t that contains the following:

$$B_t = (\llbracket \mathbf{w}_t \rrbracket_{pk_{FHE}}, [\mathbb{M}_t^1, \mathbb{M}_t^2, \dots, \mathbb{M}_t^K]) \quad (11)$$

and appends B_t to the blockchain.

5.4. Collaborative Global Model Decryption

As described in Algorithm 4, each $\mathbf{MC}_k$ downloads the generated block B_t from the blockchain. $\mathbf{MC}_k$ executes the partial decryption algorithm to obtain $\mathbf{w}_t^k$, i.e.,

$$\mathbf{w}_t^k = \llbracket \mathbf{w}_t \rrbracket_{pk_{FHE}} \cdot sk_{FHE}^k \quad (12)$$

Then, each device $\mathbf{MC}_k$ broadcasts the encrypted

$$\overline{(\mathbf{w}_t^k)_{k \rightarrow j}} = Enc_{key_{kj}}(\mathbf{w}_t^k) \quad (13)$$

for other $\mathbf{MC}_j$ ($j \neq k$). And then, upon receiving $\overline{(\mathbf{w}_t^k)_{k \rightarrow j}}$ from $\mathbf{MC}_k$, $\mathbf{MC}_j$ computes $\mathbf{w}_t^k = Dec_{key_{kj}}(\overline{(\mathbf{w}_t^k)_{k \rightarrow j}})$. Furthermore, $\mathbf{MC}_j$ randomly selects v ($v \geq m$) numbers from the set $\{1, 2, \dots, K\}$ to form a decryption set DS . Finally, $\mathbf{MC}_j$ executes the final decryption algorithm to obtain $\mathbf{w}_t$, i.e.,

$$\mathbf{w}_t = \sum_{k \in DS} \lambda_k \mathbf{w}_t^k \quad (14)$$

where λ_k ($k \in DS$) is the Lagrange coefficient.

Algorithm 4: Collab_Decryption

Input: $\mathbf{MC}_k$ holds $\llbracket \mathbf{w} \rrbracket_{pk_{FHE}}$, sk_{FHE}^k , and $\{key_{kj}\}_{j \in \{1, \dots, K\} \setminus \{k\}}$

Output: Global model $\mathbf{w}$.

- 1: $\mathbf{w}^k \leftarrow \llbracket \mathbf{w} \rrbracket_{pk} \cdot sk_{FHE}^k$;
 - 2: **for each** $i \in \{\mathbf{MC}_1, \dots, \mathbf{MC}_K\} \setminus \{\mathbf{MC}_k\}$ **do**

$\overline{(\mathbf{w}^k)_{k \rightarrow i}} \leftarrow Enc_{key_{ki}}(\mathbf{w}^k)$;
 Send $\overline{(\mathbf{w}^k)_{k \rightarrow i}}$ to $\mathbf{MC}_i$;
end
 - 3: Map $\{\mathbf{MC}_1, \dots, \mathbf{MC}_K\} \setminus \{\mathbf{MC}_k\}$ to $\{1, \dots, K-1\}$;
 - 4: Arbitrary choose $DS \subseteq \{1, \dots, K-1\}$ of size v ($v \geq m$) ;
 - 5: $\mathbf{w}_{t-1} \leftarrow 0$;
 - 6: **for each** $j \in DS$ **do**

Receive $\overline{(\mathbf{w}^j)_{j \rightarrow k}}$ from $\mathbf{MC}_j$;
 $\mathbf{w}^j \leftarrow Dec_{key_{jk}}(\overline{(\mathbf{w}^j)_{j \rightarrow k}})$;
 Computes the Lagrange coefficients λ_j ;
 $\mathbf{w} \leftarrow \mathbf{w} + \lambda_j \mathbf{w}^j$;
end
-

6. Security Analysis of PBSL

In this section, we review our security requirements of PBSL, described in Section 2, and analyze the security of PBSL.

6.1. Privacy and Confidentiality

Theorem 1. PBSL can achieve the privacy of local model gradient $\mathbf{g}_t^k$ and the confidentiality of global model $\mathbf{w}_t$.

Proof. We show that $\mathbf{g}_t^k$ and $\mathbf{w}_t$ are able to be guaranteed at different stages of swarm learning.

- In the stage of local computation, $\mathbf{g}_t^k$ is encrypted with pk_{FHE} , i.e.,

$$[\![\mathbf{g}_t^k]\!]_{pk_{FHE}} = \text{Enc}(\mathbf{g}_t^k, pk_{FHE}) \quad (15)$$

where $t = \{1, 2, \dots, T\}$ and $k = \{1, 2, \dots, K\}$, and TA splits sk_{FHE} into K distributed secret keys $sk_{FHE}^1, sk_{FHE}^2, \dots, sk_{FHE}^K$ for MCs. Because any $\mathbf{MC}_k$ with only a single sk_{FHE}^k cannot decrypt $[\![\mathbf{g}_t^k]\!]_{pk_{FHE}}$, PBSL can protect the privacy of $\mathbf{g}_t^k$ at this stage.

- At the stage of global model aggregation, the winning $\mathbf{MC}_k$ homomorphically aggregates $\{[\![\mathbf{g}_t^k]\!]_{pk_{FHE}}\}_{k=1,2,\dots,K}$ to generate $[\![\tilde{\mathbf{g}}_t]\!]_{pk_{FHE}}$, then homomorphically calculates $[\![S_t^k]\!]_{pk_{FHE}}$ by $[\![S_t^k]\!]_{pk_{FHE}} = 2 \cdot \text{ReLU}([\![\tilde{\mathbf{g}}_t]\!]_{pk_{FHE}} \odot [\![\tilde{\mathbf{g}}_t]\!]_{pk_{FHE}})' - [\![1]\!]_{pk_{FHE}}$, and updates

$$[\![\mathbf{w}_t]\!]_{pk_{FHE}} = [\![\mathbf{w}_{t-1}]\!]_{pk_{FHE}} - \gamma \frac{\sum_{k=1}^K [\![S_t^k]\!]_{pk_{FHE}} \cdot [\![\tilde{\mathbf{g}}_t]\!]_{pk_{FHE}}}{\sum_{k=1}^K [\![S_t^k]\!]_{pk_{FHE}}} \quad (16)$$

The whole process is performed on cipher-text. And $\mathbf{MC}_k$ with only a single sk_{FHE}^k cannot decrypt $[\![\mathbf{w}_t]\!]_{pk_{FHE}}$. Therefore, PBSL can protect the confidentiality of $\mathbf{w}_t$ in the stage.

- At the stage of collaborative global model decryption, all MCs decrypt the $[\![\mathbf{w}_t]\!]_{pk_{FHE}}$ distributedly, and each $\mathbf{MC}_k$ shares its partially decryption result $\mathbf{w}_t^k$. Because any $\mathbf{MC}_k$ with only a single sk_{FHE}^k cannot decrypt $[\![\mathbf{w}_t]\!]_{pk_{FHE}}$, PBSL can protect the privacy of $\mathbf{w}_t$ in the stage.

□

6.2. Byzantine-Robust

Theorem 2. For an arbitrary number of malicious MCs, in the t th iteration, the loss between the learned model $\mathbf{w}_t$ and the optimal model $\hat{\mathbf{w}}$ under no attacks is bounded, i.e.,

$$\|\mathbf{w}_t - \hat{\mathbf{w}}\| \leq (1 - \rho)^t \|\mathbf{w}^0 - \hat{\mathbf{w}}\| + 12\beta\Delta_1\rho + \text{error} \quad (17)$$

with probability at least $1 - \delta$, where error is caused by CKKS.

Proof. Our PBSL improves the encryption algorithm of FLTrust [55] and extends FLTrust to blockchain, so we directly apply FLTrust's security analysis to PBSL. According to FLTrust, in our PBSL, the loss between the learned model $\mathbf{w}_t$ in the t th iteration and the optimal model $\hat{\mathbf{w}}$ is

$$\|\mathbf{w}_t - \hat{\mathbf{w}}\| \leq (1 - \rho)^t \|\mathbf{w}^0 - \hat{\mathbf{w}}\| + 12\beta\Delta_1\rho + \text{error} \quad (18)$$

with probability at least $1 - \delta$, where error is caused by CKKS. As described in CKKS [34], Equation (19) gives the bound of the encoding and encryption errors.

$$B = 8\sqrt{2}\sigma N + 6\sigma\sqrt{N} + 16\sigma\sqrt{hN} \quad (19)$$

Therefore, error generated by the cipher-text operation of our PBSL can be expressed as follows:

$$\text{error} = B + B_1 + B_2 + v_1 B_2 + v_2 B_1 + B_1 B_2 + B_{\text{mult}(l)} \quad (20)$$

where *error* from $\oplus$ is $B_1 + B_2$, *error* from $\otimes$ is $v_1 B_2 + v_2 B_1 + B_1 B_2 + B_{mult(l)}$, $B_{mult(l)} = P^{-1} \cdot q_l \cdot B_{ks} + B_{scale}$, $B_{ks} = 8\sigma N / \sqrt{3}$, $P = P(\lambda, q_L)$, and $B_{scale} = \sqrt{N/3} \cdot (3 + 8\sqrt{h})$. Then, when $|1 - \rho| < 1$, we have $\lim_{t \rightarrow \infty} \|\mathbf{w}_t - \mathbf{w}^*\| \leq 12\beta\Delta_1/\rho + \text{error}$. $\square$

Therefore, the loss between the learned model $\mathbf{w}_t$ and the optimal model $\mathbf{w}$ is bounded.

7. Evaluation

In this section, we evaluate our scheme and present the experimental results to analyze the performance of our PBSL from two perspectives: defense effectiveness against attacks, and resource overhead in terms of computation.

7.1. Experimental Setup

7.1.1. Experimental Environment

Implementation Settings: In order to evaluate the integrated performance, we implement the PBSL scheme on a private Ethereum blockchain setup with real medical datasets. In our experiment, smart contract performs secure a global gradient aggregation algorithm instead of the centralized server.

- **A blockchain system** uses Ethereum, a blockchain-based smart contracts platform [56–58]. Smart contracts developed in the Solidity programming language are deployed to private blockchains using Truffle v5.10. And Ethereum is deployed on a desktop computer with 3.3 GHz Intel(R) Xeon(R) CPU, 16 GB memory, and Windows 10.
- **CKKS Scheme** is deployed using the TenSEAL library, which is a library for carrying out homomorphic encryption operations on tensors, and its code is available on Github (<https://github.com/OpenMined/TenSEAL>, accessed on 9 June 2024).

Dataset: Our PBSL scheme was evaluated using two widely used datasets, the MNIST dataset and the FashionMNIST dataset.

- **MNIST dataset** consists of handwritten digital pictures of 250 different people. Every sample is a black and white image with 28×28 pixels. The dataset is divided into a training set with 60,000 samples and a test set with 10,000 samples. We distributed the entire dataset evenly across 10 MCs.
- **FashionMNIST dataset** constitutes images in 10 categories. The whole dataset contains 60,000 samples (50,000 for training and 10,000 for testing). The dataset was also evenly distributed to 10 MCs.

7.1.2. Byzantine Attacks

In our experiments, the following Byzantine attacks were considered:

- **Label Flipping Attack:** By changing the label of the training sample to another category, the label flipping attacker misleads the classifier. To simulate this attack, the labels of the samples on each malicious $\mathbf{MC}_k$ are modified from l to $N - l - 1$, where N is the total number of labels and $l \in \{0, 1, \dots, N - 1\}$.
- **Backdoor Attack:** Backdoor attackers force classifiers to make bad decisions about samples with specific characteristics. To simulate the attack, we choose 6×6 pixels as the trigger, and randomly select a number of training images on each malicious MCs, and overwrite the 6×6 pixels at the top left of each selected images. Then, we reset the labels of all the selected images to the target class.
- **Arbitrary Model Attack:** The model attacker arbitrarily tampers with some of the local model parameters. To reproduce the arbitrary model attack, malicious $\mathbf{MC}_k$ select a random number as its j th local model parameter.
- **Untargeted Model Poisoning:** The model attacker constructs the toxic local model that is similar to benign local models but causes the correct global gradient to reverse, or is much bigger (or smaller) than the maximum (or minimum) benign local models.

7.1.3. Evaluation Metrics

To verify that our PSDL scheme improves model accuracy, we use **test accuracy** and **test error rate** as evaluation measures on different training datasets. In the case of different numbers of malicious MCs, the results of our PBSL scheme were verified by using the no-attack FedSGD scheme [38] as the baseline, and the Krum scheme [16], Bulyan [16], Trim-mean [39] as the control.

7.1.4. SL Settings

In our experiment, the total number K of MCs is set to 10, and all MCs are selected in each iteration during training. And we selected the models derived from convolutional neural networks (CNNs) with structure: $Input \rightarrow Conv \rightarrow FC \rightarrow Output$, to train three different datasets using Python, Numpy, and Tensorflow. Then, the training data are evenly distributed to each MC, i.e., for both MNIST and FashionMNIST datasets, the size of each local dataset is 6000. We considered the proportion of different malicious MCs from 10% to 50%. Our PBSL scheme is compared with SL and Krum. And the batch size b is set as 128.

7.2. Experimental Results

To demonstrate that the PBSL scheme meets our goals of accuracy, robustness, privacy, efficiency, and reliability, we conduct the following experiments. The experimental results show that PBSL can achieve the expected goals, which are discussed below.

7.2.1. Experimental Results on Defense Performance

Impact of Number of MCs: The test accuracy was measured by increasing the number of MCs involved in collaborative training. The experiments were run with different numbers of MCs from 1 to 10. Figure 4 shows that the more MCs participate, the higher the training accuracy.

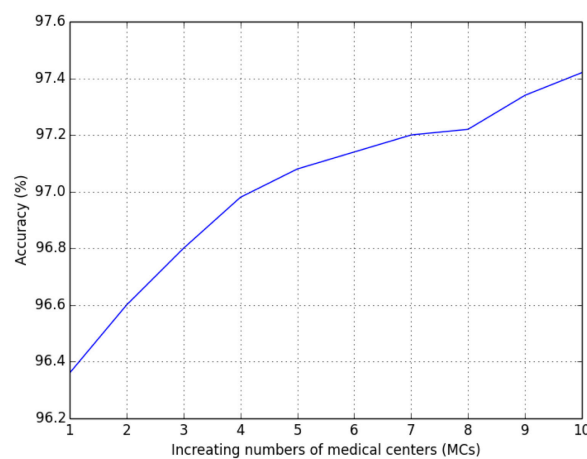


Figure 4. The impact of the number of MCs.

Impact of Different Proportions of Malicious MCs: To check the robustness of our PBSL scheme, we evaluate the accuracy loss caused by the ratio change in malicious MCs. Figures 5 and 6 show the results of PBSL training on two different datasets in the presence of different proportions of malicious MCs. The experiment shows that the proposed basic scheme is reasonable and robust.

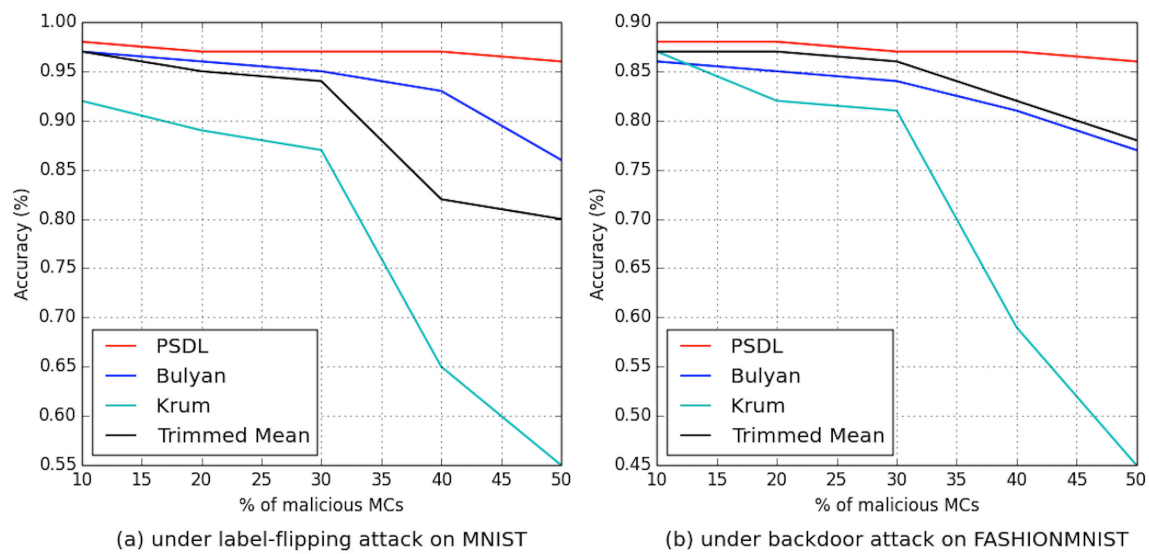


Figure 5. Comparison of accuracy with different numbers of malicious MCs under different attacks on different datasets.

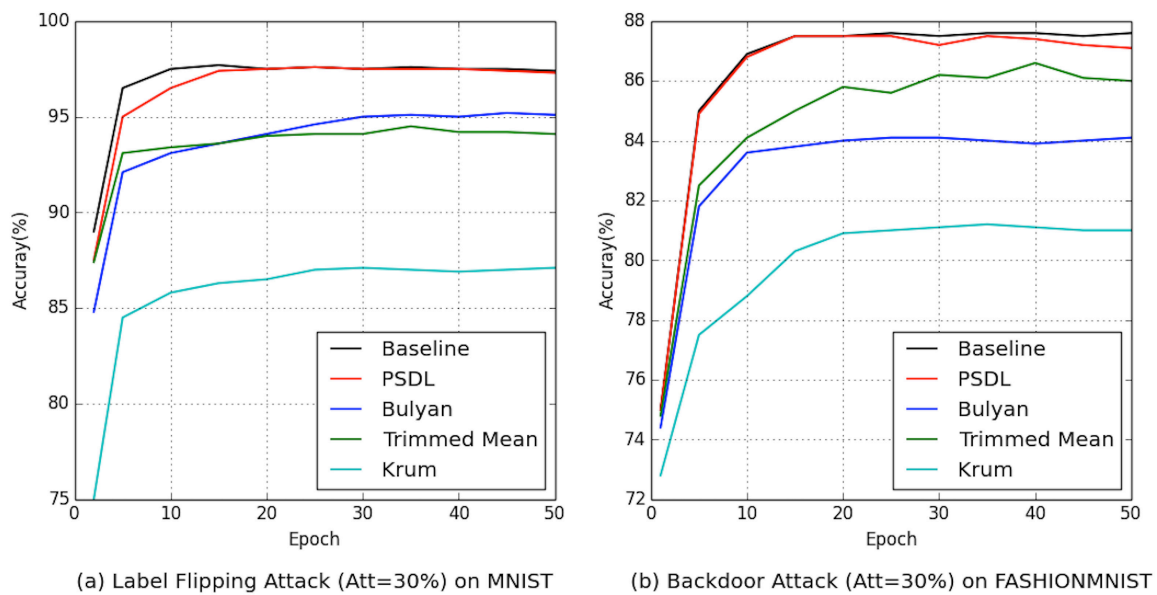


Figure 6. Comparison of accuracy with different epochs under different attacks on different datasets.

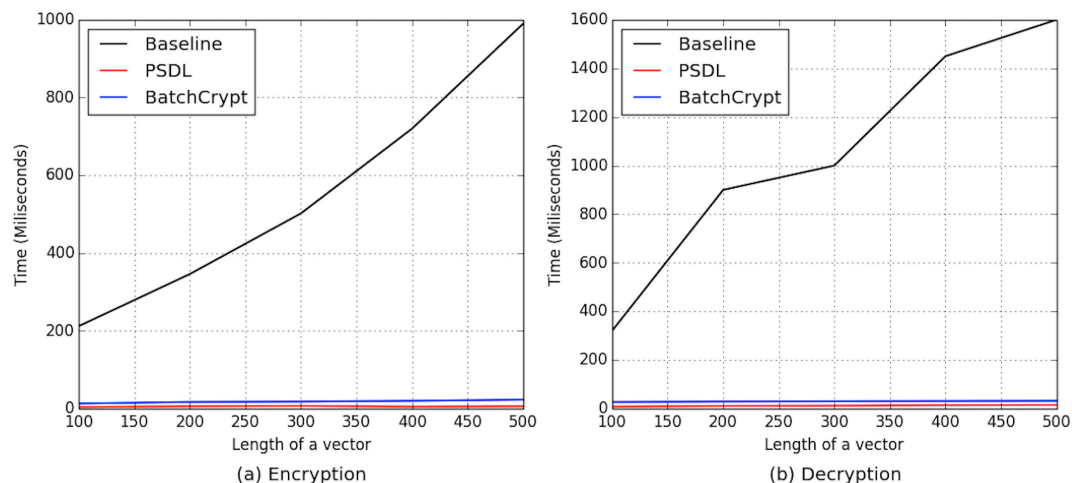
Impact of different attacks: To prove Byzantine robustness against different typical Byzantine attacks, we performed targeted attacks and untargeted attacks to our PBSL scheme. Table 3 shows the training results on MNIST and FashionMNIST datasets in the presence of different proportions of malicious MCs under targeted and untargeted attacks. The results show that PBSL still keeps the same accuracy as baseline for different typical attacks, achieving the goal of robustness and accuracy.

Table 3. Accuracy of PBSL and Bulyan with different numbers of malicious MCs under untargeted attack and targeted attack on different datasets.

Dataset	Scheme (Attacks)	% of Malicious MCs				
		10%	20%	30%	40%	50%
MNIST	PBSL (Untargeted)	98.0	98.0	97.0	97.0	96.0
	Bulyan (Untargeted)	97.0	95.0	94.0	80.0	58.0
	PBSL (Targeted)	98.0	97.0	97.0	97.0	96.0
	Bulyan (Targeted)	97.0	96.0	94.0	84.0	78.0
FashionMNIST	PBSL (Untargeted)	87.0	87.0	86.0	85.0	85.0
	Bulyan (Untargeted)	85.0	84.0	83.0	79.0	54.0
	PBSL (Targeted)	88.0	88.0	87.0	87.0	86.0
	Bulyan (Targeted)	87.0	87.0	85.0	80.0	72.0

7.2.2. Efficiency Comparison with Related FL Approaches

Comparison of PBSL with alternative privacy protection schemes. In the PBSL scheme, all operations, including similarity calculations and model aggregation, are computed in cipher-text. No participant can recover the original plaintext of the MCs' from the cipher-text. Therefore, the PBSL scheme meets the MCs' requirements for data privacy. To verify the efficiency of PBSL, using the Paillier-based federated learning scheme, i.e., PEFL [41], as a benchmark, we evaluated PBSL and BatchCrypt, where BatchCrypt [33] is a federated learning scheme based on HE. For each MC, we use the time required for encryption and decryption of each iteration as a criterion. As shown in Figure 7, PBSL greatly improves computational efficiency and reduces computational costs compared to baseline and BatchCrypt. PBSL adopts a novel encryption scheme, i.e., CKKS [34], which has its unique advantages in encrypting large-scale floating-point vectors and network models with many parameters.

**Figure 7.** Comparison of the cost time of encryption and decryption in baseline, BatchCrypt, and PBSL.

Comparison of PBSL with alternative blockchain-based schemes. Table 4 compares the model accuracy and mining overhead on the FashionMNIST dataset between LearningChain in [59], BCFL [45], and PBSL, where α denotes training time per iteration and β denotes mining time per block. First, we observe that the proposed PBSL achieves the highest model accuracy, compared with LearningChain and BCFL. This is because the privacy protection of PBSL uses fully homomorphic encryption (FHE) instead of differential

privacy, which improves the accuracy of the model. Second, the mining overhead of PBSL is less than that of LearningChain but more than that of BCFL. This is due to the fact that LearningChain defends against Byzantine attacks at the cost of resource consumption for mining, while BCFL reduces the mining overhead at the expense of model robustness.

Table 4. Comparison of accuracy and mining overhead on FashionMNIST dataset.

(α, β)	LearningChain		BCFL		PBSL	
	Accuracy	Overhead	Accuracy	Overhead	Accuracy	Overhead
(1, 10)	73.3%	100	76.5%	70	86.3%	90
(1, 8)	72.8%	94	75.7%	70	85.2%	84
(1, 6)	76.4%	98	77.5%	74	87.4%	88
(2, 10)	67.9%	90	72.2%	52	82.1%	80
(5, 10)	51.8%	70	56.5%	46	74.2%	60

And the model aggregation is implemented as functions of smart contract, and the computational cost of a function in a smart contract is assessed in gas units. Therefore, we measure the computational consumption of PBSL in gas. Using the CNN model, gas costs are tested in each iteration with 10 MCs. After all the MCs have submitted their local updates in cipher-text, SC obtains the global gradient at 30,500,242 gas per transaction by homomorphically aggregating all local updates.

8. Conclusions

In the article, we review the shortcomings of swarm learning (SL) and further propose a new privacy-protection Byzantine-resilient swarm learning for e-health, called PBSL, which ensures the privacy of local gradient and the tolerance toward Byzantine attacks. With the CKKS cryptosystem based on threshold decryption and decentralized computing based on blockchain, each MC can safely aggregate other MCs' gradients to generate a more accurate global model, while the confidentiality of the final global model can be ensured. And in the process of global model aggregation, our proposed Byzantine-robust aggregation method uses secure cosine similarity to punish malicious MCs. Detailed security analysis and extensive experiments show that PBSL has privacy protection capability and efficiency.

In this paper, our attack detection strategy is based on cosine similarity between the local model gradient and the global model gradient. In future work, in order to prevent new attack vectors, we will continue to study new attack detection strategies that are suitable for blockchain platforms and take into account privacy protection, so as to deal with unforeseen threats in future-proofing systems.

Author Contributions: Conceptualization, X.Z.; Methodology, X.Z.; Software, T.L.; Writing—original draft, X.Z.; Writing—review & editing, T.L.; Supervision, H.L.; Project administration, H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Publicly available datasets were analyzed in this study. This data can be found here: [<http://yann.lecun.com/exdb/mnist/>, <https://github.com/zalandoresearch/fashion-mnist>].

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Meier, C.; Fitzgerald, M.C.; Smith, J.M. Ehealth: Extending, enhancing, and evolving health care. *Annu. Rev. Biomed. Eng.* **2013**, *15*, 359–382. [[CrossRef](#)] [[PubMed](#)]
2. Liu, X.; Lu, R.; Ma, J.; Chen, L.; Qin, B. Privacy-preserving patient-centric clinical decision support system on naive bayesian classification. *IEEE J. Biomed. Health Inform.* **2016**, *20*, 655–668. [[CrossRef](#)] [[PubMed](#)]

3. Rahulamathavan, Y.; Veluru, S.; Phan, C.W.; Chambers, J.A.; Rajarajan, M. Privacy-preserving clinical decision support system using gaussian kernel-based classification. *IEEE J. Biomed. Health Inform.* **2014**, *18*, 56–66. [[CrossRef](#)] [[PubMed](#)]
4. Wiens, J.; Saria, S.; Sendak, M.; Ghassemi, M.; Liu, V.X.; Doshi-Velez, F.; Jung, K.; Heller, K.; Kale, D.; Saeed, M.; et al. Do no harm: A roadmap for responsible machine learning for healthcare. *Nat. Med.* **2019**, *2019*, 1337–1340. [[CrossRef](#)] [[PubMed](#)]
5. Courtiol, P.; Maussion, C.; Moarii, M.; Pronier, E.; Pilcer, S.; Sefta, M.; Manceron, P.; Toldo, S.; Zaslavskiy, M.; Le Stang, N.; et al. Deep learning-based classification of mesothelioma improves prediction of patient outcome. *Nat. Med.* **2019**, *25*, 1519–1525. [[CrossRef](#)] [[PubMed](#)]
6. Warnat-Herresthal, S.; Perrakis, K.; Taschler, B.; Becker, M.; Baßler, K.; Beyer, M.; Günther, P.; Schulte-Schrepping, J.; Seep, L.; Klee, K.; et al. Scalable Prediction of Acute Myeloid Leukemia Using High-Dimensional Machine Learning and Blood Transcriptomics. *iScience* **2020**, *23*, 100780. [[CrossRef](#)] [[PubMed](#)]
7. Rajkomar, A.; Dean, J.; Kohane, I. Machine learning in medicine. *N. Engl. J. Med.* **2019**, *380*, 1347–1358. [[CrossRef](#)] [[PubMed](#)]
8. Savage, N. Calculating disease. *Nature* **2017**, *550*, 115–117. [[CrossRef](#)] [[PubMed](#)]
9. Ping, P.; Hermjakob, H.; Polson, J.S.; Benos, P.V.; Wang, W. Biomedical informatics on the cloud: A treasure hunt for advancing cardiovascular medicine. *Circ. Res.* **2018**, *122*, 1290–1301. [[CrossRef](#)]
10. Kaissis, G.A.; Makowski, M.R.; Ruckert, D.; Braren, R.F. Secure privacy-preserving and federated machine learning in medical imaging. *Nat. Mach. Intell.* **2020**, *2*, 305–311. [[CrossRef](#)]
11. Konecny, J.; McMahan, H.B.; Yu, F.X.; Richtárik, P.; Suresh, A.T.; Bacon, D. Federated learning: Strategies for improving communication efficiency. *arXiv* **2016**, arXiv:1610.05492.
12. McMahan, H.B.; Moore, E.; Ramage, D.; Arcas, B. Federated learning of deep networks using model averaging. *arXiv* **2016**, arXiv:1602.05629v1.
13. Warnat-Herresthal, S.; Schultze, H.; Shastri, K.L.; Manamohan, S.; Mukherjee, S.; Garg, V.; Sarveswara, R.; Händler, K.; Pickkers, P.; Aziz, N.A.; et al. Swarm Learning for decentralized and confidential clinical machine learning. *Nature* **2021**, *594*, 265–270. [[CrossRef](#)] [[PubMed](#)]
14. Song, C.; Ristenpart, T.; Shmatikov, V. Machine learning models that remember too much. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Dallas, TX, USA, 30 October–3 November 2017; ACM: New York, NY, USA, 2017; pp. 587–601.
15. Melis, L.; Song, C.; De Cristofaro, E.; Shmatikov, V. Inference attacks against collaborative learning. *arXiv* **2018**, arXiv:1805.04049.
16. Blanchard, P.; El Mhamdi, E.M.; Guerraoui, R.; Stainer, J. Machine learning with adversaries: Byzantine tolerant gradient descent. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 119–129.
17. Bagdasaryan, E.; Veit, A.; Hua, Y.; Estrin, D.; Shmatikov, V. How to backdoor federated learning. *arXiv* **2018**, arXiv:1807.00459.
18. Shokri, R.; Shmatikov, V. Privacy-preserving deep learning. In Proceedings of the ACM Conference on Computer and Communications Security, Denver, CO, USA, 12–16 October 2015; pp. 310–321.
19. Heikkilä, M.A.; Koskela, A.; Shimizu, K.; Kaski, S.; Honkela, A. Differentially private cross-silo federated learning. *arXiv* **2020**, arXiv:2007.05553.
20. Zhao, L.; Wang, Q.; Zou, Q.; Zhang, Y.; Chen, Y. Privacy-preserving collaborative deep learning with unreliable participants. *IEEE Trans. Inf. Forensics Secur.* **2020**, *15*, 1486–1500. [[CrossRef](#)]
21. Xu, R.; Baracaldo, N.; Zhou, Y.; Anwar, A.; Ludwig, H. HybridALpha: An efficient approach for privacy-preserving federated learning. In Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security, London, UK, 15 November 2019; pp. 13–23.
22. Hinton, G.; Deng, L.; Yu, D.; Dahl, G.E.; Mohamed, A.R.; Jaitly, N.; Senior, A.; Vanhoucke, V.; Nguyen, P.; Sainath, T.N.; et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Process. Mag.* **2012**, *29*, 82–97. [[CrossRef](#)]
23. Chan, T.-H.; Jia, K.; Gao, S.; Lu, J.; Zeng, Z.; Ma, Y. Pcanet: A simple deep learning baseline for image classification? *IEEE Trans. Image Process.* **2015**, *24*, 5017–5032. [[CrossRef](#)]
24. Aliper, A.; Plis, S.; Artemov, A.; Ulloa, A.; Mamoshina, P.; Zhavoronkov, A. Deep learning applications for predicting pharmacological properties of drugs and drug repurposing using transcriptomic data. *Mol. Pharm.* **2016**, *13*, 2524–2530. [[CrossRef](#)]
25. Budaher, J.; Almasri, M.; Goeuriot, L. Comparison of several word embedding sources for medical information retrieval. In Proceedings of the Working Notes of CLEF 2016—Conference and Labs of the Evaluation Forum, Évora, Portugal, 5–8 September 2016; pp. 43–46.
26. Rav, D.; Wong, C.; Deligianni, F.; Berthelot, M.; Andreu-Perez, J.; Lo, B.; Yang, G.Z. Deep learning for health informatics. *IEEE J. Biomed. Health Inf.* **2017**, *21*, 4–21. [[CrossRef](#)]
27. Bonawitz, K.; Ivanov, V.; Kreuter, B.; Marcedone, A.; McMahan, H.B.; Patel, S.; Ramage, D.; Segal, A.; Seth, K. Practical secure aggregation for privacy-preserving machine learning. In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, Dallas, TX, USA, 30 October–3 November 2017; pp. 1175–1191.
28. Mohassel, P.; Zhang, Y. Secureml: A system for scalable privacy-preserving machine learning. In Proceedings of the 2017 IEEE Symposium on Security and Privacy (SP), San Jose, CA, USA, 25 May 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 19–38.
29. Wagh, S.; Gupta, D.; Chandran, N. Securenn: 3-party secure computation for neural network training. *Proc. Priv. Enhanc. Technol.* **2019**, *3*, 26–49. [[CrossRef](#)]

30. Chaudhari, H.; Rachuri, R.; Suresh, A. Trident: Efficient 4pc framework for privacy preserving machine learning. In Proceedings of the 27th Annual Network and Distributed System Security Symposium, NDSS, San Diego, CA, USA, 23–26 February 2020.
31. Phong, L.T.; Aono, Y.; Hayashi, T.; Wang, L.; Moriai, S. Privacy-preserving deep learning via additively homomorphic encryption. *IEEE Trans. Inf. Forensics Secur.* **2018**, *13*, 1333–1345. [\[CrossRef\]](#)
32. Dong, Y.; Chen, X.; Shen, L.; Wang, D. EaSTFLy: Efficient and secure ternary federated learning. *Comput. Secur.* **2020**, *94*, 101824. [\[CrossRef\]](#)
33. Zhang, C.; Li, S.; Xia, J.; Wang, W. BatchCrypt: Efficient homomorphic encryption for Cross-Silo federated learning. In Proceedings of the USENIX Annual Technical Conference (USENIX ATC 20), Online, 15–17 July 2020; pp. 493–506.
34. Cheon, J.; Kim, A.; Kim, M.; Song, Y. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology, Proceedings of the ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, 3–7 December 2017*; Springer: Cham, Switzerland, 2017; pp. 409–437.
35. Fung, C.; Yoon, C.J.M.; Beschastnikh, I. Mitigating sybils in federated learning poisoning. *arXiv* **2018**, arXiv:1808.04866.
36. Fang, M.; Cao, X.; Jia, J.; Gong, N. Local model poisoning attacks to byzantine-robust federated learning. In Proceedings of the 29th USENIX Security Symposium (USENIX Security), Boston, MA, USA, 12–14 August 2020; pp. 1605–1622.
37. Tolpegin, V.; Truex, S.; Gursoy, M.E.; Liu, L. Data poisoning attacks against federated learning systems. In Proceedings of the 25th European Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, 14–18 September 2020; Springer: Cham, Switzerland, 2020; pp. 480–501.
38. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; Arcas, B.A.Y. Communication-efficient learning of deep networks from decentralized data. In Proceedings of the International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 20–22 April 2017; pp. 1273–1282.
39. Yin, D.; Chen, Y.; Ramchandran, K.; Bartlett, P.L. Byzantine-robust distributed learning: Towards optimal statistical rates. In Proceedings of the International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018; pp. 5636–5645.
40. Truex, S.; Baracaldo, N.; Anwar, A.; Steinke, T.; Ludwig, H.; Zhang, R.; Zhou, Y. A hybrid approach to privacy-preserving federated learning. In Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security, London, UK, 15 November 2019; pp. 1–11.
41. Liu, X.; Li, H.; Xu, G.; Chen, Z.; Huang, X.; Lu, R. Privacy-enhanced federated learning against poisoning adversaries. *IEEE Trans. Inf. Forensics Secur.* **2021**, *16*, 4574–4588. [\[CrossRef\]](#)
42. Chen, Y.; Su, L.; Xu, J. Distributed statistical machine learning in adversarial settings: Byzantine gradient descent. *Proc. ACM Meas. Anal. Comput. Syst.* **2017**, *1*, 1–25. [\[CrossRef\]](#)
43. Guerraoui, R.; Rouault, S. The hidden vulnerability of distributed learning in byzantium. In Proceedings of the International Conference on Machine Learning (ICML 2018), Stockholm, Sweden, 10–15 July 2018; pp. 3521–3530.
44. Ramanan, P.; Nakayama, K. BAFFLE: Blockchain based aggregator free federated learning. In Proceedings of the International Conference on Blockchain (Blockchain), Virtual Event, 2–6 November 2020; pp. 72–81.
45. Li, Y.; Chen, C.; Liu, N.; Huang, H.; Zheng, Z.; Yan, Q. A blockchain-based decentralized federated learning framework with committee consensus. *IEEE Netw.* **2021**, *35*, 234–241. [\[CrossRef\]](#)
46. Kim, H.; Park, J.; Bennis, M.; Kim, S.L. Blockchained on-device federated learning. *IEEE Commun. Lett.* **2020**, *24*, 1279–1283. [\[CrossRef\]](#)
47. Weng, J.; Zhang, J.; Li, M.; Zhang, Y.; Luo, W. DeepChain: Auditable and Privacy-Preserving Deep Learning with Blockchain-Based Incentive. *IEEE Trans. Dependable Secur. Comput.* **2021**, *18*, 2438–2455. [\[CrossRef\]](#)
48. Warnat-Herresthal, S.; Schultze, H.; Shastri, K.; Manamohan, S.; Mukherjee, S.; Garg, V.; Sarveswara, R.; Händler, K.; Pickkers, P.; Aziz, N.A.; et al. Swarm Learning as a privacy-preserving machine learning approach for disease classification. *bioRxiv* **2020**. [\[CrossRef\]](#)
49. Fan, D.; Wu, Y.; Li, X. On the Fairness of Swarm Learning in Skin Lesion Classification. In *Clinical Image-Based Procedures, Distributed and Collaborative Learning, Artificial Intelligence for Combating COVID-19 and Secure and Privacy-Preserving Machine Learning, Proceedings of the 10th Workshop, CLIP 2021, Second Workshop, DCL 2021, First Workshop, LL-COVID19 2021, and First Workshop and Tutorial, PPML 2021, Strasbourg, France, 27 September–1 October 2021*; CoRR Abs/2109.12176; Springer: Berlin/Heidelberg, Germany, 2021.
50. Oestreich, M.; Chen, D.; Schultze, J.; Fritz, M.; Becker, M. Privacy considerations for sharing genomics data. *Excli J.* **2021**, *2021*, 1243.
51. Westerlund, A.; Hawe, J.; Heinig, M.; Schunkert, H. Risk Prediction of Cardiovascular Events by Exploration of Molecular Data with Explainable Artificial Intelligence. *Int. J. Mol. Sci.* **2021**, *22*, 10291. [\[CrossRef\]](#) [\[PubMed\]](#)
52. Jain, A.; Rasmussen, P.M.R.; Sahai, A. Threshold Fully Homomorphic Encryption. Cryptology ePrint Archive, Report 2017/257, 2017. Available online: <http://eprint.iacr.org/2017/257> (accessed on 9 June 2024).
53. Paillier, P. Public-key crypto-systems based on composite degree residuosity classes. In Proceedings of the International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, 1–2 May 1999; pp. 223–238.
54. Cheon, J.; Kim, D.; Lee, H.; Lee, K. Numerical method for comparison on homomorphically encrypted numbers. In Proceedings of the ASIACRYPT 2019 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, 8–12 December 2019; pp. 415–445.

55. Cao, X.; Fang, M.; Liu, J.; Gong, N.Z. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In Proceedings of the Network and Distributed System Security Symposium, Virtual, 21–25 February 2021; pp. 1–18. [[CrossRef](#)]
56. Li, Z.; Kang, J.; Yu, R.; Ye, D.; Deng, Q.; Zhang, Y. Consortium blockchain for secure energy trading in industrial internet of things. *IEEE Trans. Ind. Inform.* **2018**, *14*, 3690–3700. [[CrossRef](#)]
57. Hu, S.; Cai, C.; Wang, Q.; Wang, C.; Luo, X.; Ren, K. Searching an encrypted cloud meets blockchain: A decentralized, reliable and fair realization. In Proceedings of the IEEE INFOCOM 2018-IEEE Conference on Computer Communications, Honolulu, HI, USA, 16–19 April 2018.
58. Wang, S.; Zhang, Y.; Zhang, Y. A blockchain-based framework for data sharing with fine-grained access control in decentralized storage systems. *IEEE Access* **2018**, *6*, 437–450. [[CrossRef](#)]
59. Chen, X.; Luo, J.J.; Liao, C.W.; Li, P. When machine learning meets blockchain: A decentralized, privacy-preserving and secure design. In Proceedings of the 2018 IEEE International Conference on Big Data (Big Data), Seattle, WA, USA, 10–13 December 2018; pp. 1178–1187.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.